Початок роботи з C# і ASP.NET Core в Visual Studio. Razor Pages (частина 3)

Завдання

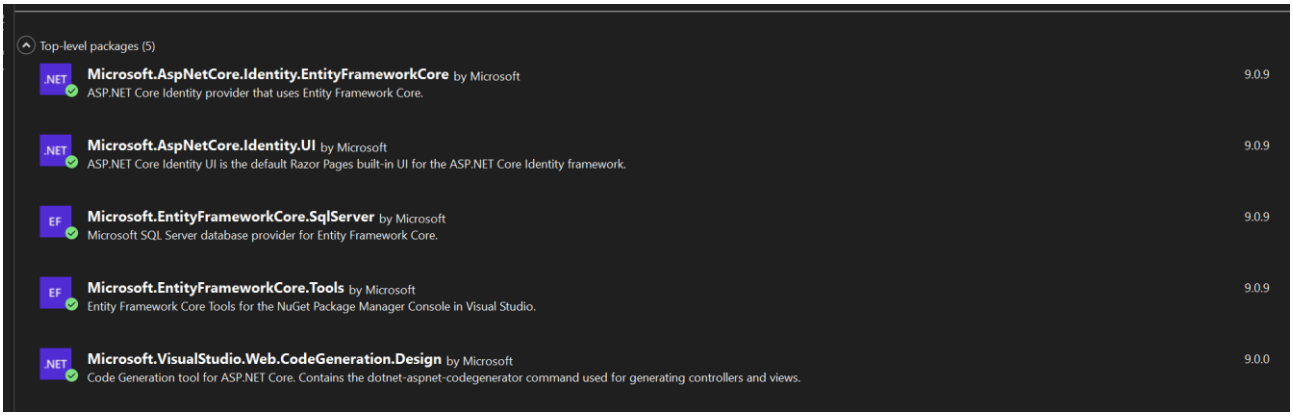
Додайте до вашої програми, створеної в результаті виконання попередніх лабораторних робіт функціональність, яка описана далі (авторизація, реєстрація, завантаження зображень, рейтинги і коментарі від користувачів).

Автентифікація та авторизація

Додаємо автентифікацію/авторизацію та роль Admin і закриваємо усі сторінки керування фільмами лише для адміністратора

Package Manager Console

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.AspNetCore.Identity.UI
```



Створіть контекст для Identity

Ми лишаємо існуючий **RazorPagesMovieContext** для фільмів, і **додаємо окремий ApplicationDbContext** для облікових записів.

Data/ApplicationDbContext.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace RazorPagesMovie.Data
```

```
{
    public class ApplicationDbContext : IdentityDbContext<IdentityUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options) { }
    }
}
```

Налаштуйте Program.cs

- Підключаємо обидва контексти до однієї БД.
- Вмикаємо автентифікацію/авторизацію.
- Закриваємо папку /Movies політикою “тільки Admin”.

Program.cs (важливі фрагменти)

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;

var builder = WebApplication.CreateBuilder(args);

// Контекст фільмів (у вас вже є)
builder.Services.AddDbContext<RazorPagesMovieContext>(options =>
  options.UseSqlServer(
    builder.Configuration.GetConnectionString("RazorPagesMovieContext")));
// Якщо SQLite, то:
// options.UseSqlite(builder.Configuration.GetConnectionString("RazorPagesMovieContext"))

// Контекст Identity (та сама БД/рядок)
builder.Services.AddDbContext<ApplicationDbContext>(options =>
  options.UseSqlServer(
    builder.Configuration.GetConnectionString("RazorPagesMovieContext")));
// або UseSqlite(...), якщо у вас SQLite

// Identity + Ролі
// Identity з ролями + вбудований UI
builder.Services.AddIdentity<IdentityUser, IdentityRole>(options =>
{
  options.SignIn.RequireConfirmedAccount = false; // для демо
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders()
.AddDefaultUI();

// Політика "лише Admin"
builder.Services.AddAuthorization(options =>
{
  options.AddPolicy("RequireAdministratorRole", policy => policy.RequireRole("Admin"));
});

// Закриваємо папку /Movies
builder.Services.AddRazorPages(options =>
```

```
{
   options.Conventions.AuthorizeFolder("/Movies", "RequireAdministratorRole");
});

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
   app.UseExceptionHandler("/Error");
   app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();   // <— важливо
app.UseAuthorization();    // <— важливо

app.MapRazorPages();

// ВАЖЛИВО: сідер ДО app.Run()
using (var scope = app.Services.CreateScope())
{
   await IdentitySeeder.SeedAdminAsync(scope.ServiceProvider, app.Configuration);
}
app.Run();
```

Додайте сідер для ролі/адміна

Сідер (додає роль і користувача, або лише роль, якщо юзер вже існує)

Data/IdentitySeeder.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace RazorPagesMovie.Data
{
    public static class IdentitySeeder
    {
        public static async Task SeedAdminAsync(IServiceProvider services,
IConfiguration cfg)
        {
            var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
            var userManager = services.GetRequiredService<UserManager<IdentityUser>>();

            var email = cfg["Admin:Email"] ?? "admin@example.com";
            var pass = cfg["Admin:Password"] ?? "Admin#12345";
            const string role = "Admin";
```

```
            if (!await roleManager.RoleExistsAsync(role))
                await roleManager.CreateAsync(new IdentityRole(role));

            var user = await userManager.FindByEmailAsync(email);
            if (user == null)
            {
                user = new IdentityUser { UserName = email, Email = email,
EmailConfirmed = true };
                var create = await userManager.CreateAsync(user, pass);
                if (!create.Succeeded)
                    throw new Exception("Admin create failed: " +
                        string.Join("; ", create.Errors.Select(e =>
$"{e.Code}:{e.Description}")));
            }

            if (!await userManager.IsInRoleAsync(user, role))
                await userManager.AddToRoleAsync(user, role);
        }
    }
}
```

Сконфігуруйте адміна в appsettings.json

```
"Admin": {
  "Email": "admin@example.com",
  "Password": "Admin#12345"
}
```

Створіть файл _LoginPartial.cshtml

Створіть **Pages/Shared/_LoginPartial.cshtml** (саме в Pages/Shared, це один із шляхів, де Identity його шукає).

Вміст (мінімальна і водночас корисна версія):

cshtml

```
@* Pages/Shared/_LoginPartial.cshtml *@
@using Microsoft.AspNetCore.Identity
@inject SignInManager<IdentityUser> SignInManager
@inject UserManager<IdentityUser> UserManager

<ul class="navbar-nav">
@if (SignInManager.IsSignedIn(User))
{
    <li class="nav-item">
        <a class="nav-link" asp-area="Identity" asp-page="/Account/Manage/Index">
            Привіт, @User.Identity?.Name
        </a>
    </li>
    <li class="nav-item">
        <form class="form-inline" asp-area="Identity" asp-page="/Account/Logout"
method="post">
            <button type="submit" class="nav-link btn btn-link">Вийти</button>
        </form>
    </li>
```

```
}
else
{
    <li class="nav-item">
        <a class="nav-link" asp-area="Identity" asp-page="/Account/Login">Увійти</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" asp-area="Identity" asp-
page="/Account/Register">Реєстрація</a>
    </li>
}
</ul>
```

За бажання — можна поки зробити «пустий» partial (без розмітки), але краще одразу мати робочі посилання.

Тепер усі сторінки в папці **Pages/Movies** доступні лише користувачам з роллю **Admin** (анонімний і звичайний користувачі їх не побачать). Це означає, що сторінки створення/редагування/видалення теж автоматично захищені.

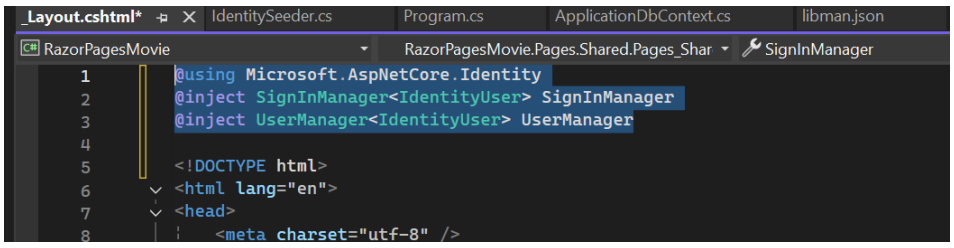
Використайте partial і у вашому _Layout
Додайте посилання Вхід/Вихід у навбар

Якщо у вашому _Layout.cshtml ще немає посилань на сторінки входу/виходу — додайте:
Pages/Shared/_Layout.cshtml (у <ul class="navbar-nav"> справа)

```
@using Microsoft.AspNetCore.Identity
@inject SignInManager<IdentityUser> SignInManager
@inject UserManager<IdentityUser> UserManager

<ul class="navbar-nav flex-grow-1">
  <!-- ваші інші пункти меню -->
</ul>

<partial name="_LoginPartial" />
```



```
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="navbar-collapse collapse d-sm-inline-flex justify-content-b
                <ul class="navbar-nav flex-grow-1">
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-page="/Index"
                    </li>
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-page="/Privac
                    </li>
                </ul>
                <partial name="_LoginPartial" />
            </div>
        </div>
    </nav>
</header>
<div class="container">
    <main role="main" class="pb-3">
        @RenderBody()
    </main>
</div>
```

Сторінки за замовчуванням /Identity/Account/Login, /Register, /Logout надає **Microsoft.AspNetCore.Identity.UI** — ми їх не генеруємо вручну.

Міграції
Тепер у вас **два контексти** — потрібно прогнати міграції **для кожного** (разово):

```
# якщо ще не встановлено
dotnet tool install --global dotnet-ef

# Міграції для Identity
dotnet ef migrations add CreateIdentitySchema -c ApplicationDbContext -o
Data/Migrations/Identity
dotnet ef database update -c ApplicationDbContext

# (Якщо ще не робили) Міграції для фільмів
dotnet ef migrations add InitialCreate -c RazorPagesMovieContext -o
Data/Migrations/Movies
dotnet ef database update -c RazorPagesMovieContext
```

```
Package Manager Console
Package source: All    Default project: RazorPagesMovie
PM> dotnet tool install --global dotnet-ef
You can invoke the tool using the following command: dotnet-ef
Tool 'dotnet-ef' (version '9.0.9') was successfully installed.
PM> dotnet ef migrations add CreateIdentitySchema -c ApplicationDbContext -o Data/Migrations/Identity
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'
PM> dotnet ef database update -c ApplicationDbContext
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (10ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Migrations[20411]
      Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more
information if this takes too long.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more
information if this takes too long.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (15ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @result int;
```

Якщо EF повідомляє про «pending model changes» — створіть ще одну міграцію для відповідного контексту з новою назвою і повторіть update.

Перевірте Tag Helpers
У **Pages/_ViewImports.cshtml** має бути:

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
(У типовому шаблоні це вже є.)

Перевірка

- Запустіть застосунок.
- Зайдіть як **admin@example.com / Admin#12345**.
- Переконайтеся, що сторінки /Movies (список/створення/редагування/видалення) доступні.
- Вийдіть з акаунта — /Movies тепер має робити редірект на логін.

На цьому етапі **адміністратор** керує каталогом, а звичайні користувачі не бачать службового списку фільмів.

Зробимо **публічний каталог** (доступний усім), а адмінське керування лишаємо в /Movies.

Створюємо сторінку-індекс каталогу Pages/Catalog/Index.cshtml.cs

```
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Pages.Catalog
{
    [AllowAnonymous] // публічна сторінка
    public class IndexModel : PageModel
    {
        private readonly RazorPagesMovieContext _context;
        public IndexModel(RazorPagesMovieContext context) => _context = context;

        public IList<Movie> Movies { get; private set; } = new List<Movie>();

        // Пошук за назвою (опційно)
        [Microsoft.AspNetCore.Mvc.BindProperty(SupportsGet = true)]
        public string? q { get; set; }

        public async Task OnGetAsync()
        {
            var query = _context.Movie.AsNoTracking();

            if (!string.IsNullOrWhiteSpace(q))
            {
                query = query.Where(m => m.Title.Contains(q));
            }

            Movies = await query
                .OrderBy(m => m.Title)
```

```
                .ToListAsync();
        }
    }
}
```

Pages/Catalog/Index.cshtml

```
@page
@model RazorPagesMovie.Pages.Catalog.IndexModel
@{
    ViewData["Title"] = "Каталог фільмів";
}

<h1 class="mb-3">@ViewData["Title"]</h1>

<form method="get" class="row g-2 mb-3">
    <div class="col-12 col-sm-8 col-md-6">
        <input name="q" value="@Model.q" class="form-control" placeholder="Пошук за
назвою..." />
    </div>
    <div class="col">
        <button class="btn btn-primary">Знайти</button>
        <a class="btn btn-outline-secondary" asp-page="./Index">Скинути</a>
    </div>
</form>

@if (Model.Movies.Count == 0)
{
    <div class="alert alert-info">Нічого не знайдено.</div>
}
else
{
    <div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-4 g-4">
        @foreach (var m in Model.Movies)
        {
            <div class="col">
                <div class="card h-100 shadow-sm border-0">
                    <img
                        src="~/images/placeholder-poster.svg"
                        class="card-img-top"
                        alt="Постер @m.Title"
                        style="object-fit:cover; aspect-ratio:2/3;" />
                    <div class="card-body d-flex flex-column">
                        <h5 class="card-title mb-1">@m.Title</h5>
                        <div class="text-muted small">@m.Genre</div>
                        <div class="mt-2 small">
                            Дата релізу:
                            <strong>@m.ReleaseDate.ToString("yyyy-MM-dd")</strong>
                        </div>
                        @* Якщо у моделі є Price *@
                        @if (m.GetType().GetProperty("Price") != null)
                        {
                            <div class="mt-1 fw-semibold">
                                Ціна: @((m as dynamic).Price?.ToString("C"))
                            </div>
                        }
```

```
                        <div class="mt-auto pt-2">
                            <a class="stretched-link" asp-page="./Details" asp-route-
id="@m.Id">Детальніше</a>
                        </div>
                    </div>
                </div>
            </div>
        }
    </div>
}
```

Сторінка деталей фільму (публічна)
Pages/Catalog/Details.cshtml.cs

```
using System.Threading.Tasks;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Pages.Catalog
{
    [AllowAnonymous]
    public class DetailsModel : PageModel
    {
        private readonly RazorPagesMovieContext _context;
        public DetailsModel(RazorPagesMovieContext context) => _context = context;

        public Movie? Movie { get; private set; }

        public async Task<IActionResult> OnGetAsync(int id)
        {
            Movie = await _context.Movie.AsNoTracking().FirstOrDefaultAsync(m => m.Id
== id);
            if (Movie == null) return NotFound();
            return Page();
        }
    }
}
```

Pages/Catalog/Details.cshtml

```
@page "{id:int}"
@model RazorPagesMovie.Pages.Catalog.DetailsModel
@{
    ViewData["Title"] = Model.Movie?.Title ?? "Фільм";
}

<a asp-page="./Index" class="btn btn-link ps-0">&larr; До каталогу</a>

<div class="row g-4 align-items-start">
```

```
    <div class="col-12 col-md-4 col-lg-3">
        <img
            src="~/images/placeholder-poster.svg"
            alt="Постер @Model.Movie?.Title"
            class="img-fluid rounded shadow-sm"
            style="object-fit:cover; aspect-ratio:2/3;" />
    </div>
    <div class="col">
        <h1 class="mb-2">@Model.Movie?.Title</h1>
        <div class="text-muted mb-3">@Model.Movie?.Genre</div>

        <dl class="row">
            <dt class="col-sm-3">Дата релізу</dt>
            <dd class="col-sm-9">@Model.Movie?.ReleaseDate.ToString("yyyy-MM-dd")</dd>

            @* Якщо є Price у моделі *@
            @if (Model.Movie?.GetType().GetProperty("Price") != null)
            {
                <dt class="col-sm-3">Ціна</dt>
                <dd class="col-sm-9">@((Model.Movie as
dynamic).Price?.ToString("C"))</dd>
            }

            @* Якщо є Rating у моделі *@
            @if (Model.Movie?.GetType().GetProperty("Rating") != null)
            {
                <dt class="col-sm-3">Рейтинг (текст)</dt>
                <dd class="col-sm-9">@((Model.Movie as dynamic).Rating)</dd>
            }
        </dl>

        @* Тут пізніше додамо: фото з БД/файлів, оцінювання, коментарі *@
    </div>
</div>
```

Додаємо пункт меню «Каталог»
У **Pages/Shared/_Layout.cshtml** (у <ul class="navbar-nav flex-grow-1">) додайте:

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-page="/Catalog/Index">Каталог</a>
</li>
```

Додаємо «заглушку» постера
Створіть файл **wwwroot/images/placeholder-poster.svg** з простим SVG (легкий та різкий у 2:3):

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 600 900">
  <defs>
    <linearGradient id="g" x1="0" x2="1">
      <stop offset="0" stop-color="#e9ecef"/>
      <stop offset="1" stop-color="#f8f9fa"/>
```

```
    </linearGradient>
  </defs>
  <rect width="600" height="900" fill="url(#g)"/>
  <g fill="#adb5bd" font-family="Segoe UI, Roboto, sans-serif" text-anchor="middle">
    <rect x="150" y="250" width="300" height="300" rx="20" fill="#dee2e6"/>
    <text x="300" y="600" font-size="36">Постер відсутній</text>
  </g>
</svg>
```

Перевіряємо доступи

- /Movies — **лише Admin** (як налаштовано в Кроці 1).
- /Catalog і /Catalog/Details/{id} — доступні всім (анонімним), завдяки [AllowAnonymous] та відсутності обмежувальних конвенцій на цю папку.

Змінимо верхнє меню

Лого має відкривати публічний каталог

В _Layout.cshtml Знайдіть рядок з брендом (у вас було href="/Movies") й замініть на:

```
<a class="navbar-brand" asp-page="/Catalog/Index">Каталог фільмів</a>
```

Ліве меню: показувати /Movies тільки адміну, прибрати Privacy

Знайдіть блок

```
<ul class="navbar-nav flex-grow-1">
                         <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-
page="/Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-
page="/Privacy">Privacy</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-
page="/Catalog/Index">Каталог</a>
                        </li>
</ul>
```

І замініть на

```
<ul class="navbar-nav flex-grow-1">
                         <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-
page="/Index">Головна сторінка</a>
                        </li>
```

```
    @* Лінк на адмінський розділ фільмів — лише для Admin *@
    @if (User.IsInRole("Admin"))
    {
        <li class="nav-item">
            <a class="nav-link text-dark" asp-page="/Movies/Index">Movies (Admin)</a>
        </li>
    }
</ul>
```

Це також прибирає пункт **Privacy** з меню. (Сторінку поки не чіпаємо)

3) Праворуч — залиште логін/реєстрацію

Переконайтесь, що праворуч у навбарі викликається:

```
<partial name="_LoginPartial" />
```

На сторінці /Catalog додамо фільтрування за жанром

Оновіть Pages/Catalog/Index.cshtml.cs

Додайте властивості для вибраного жанру та списку жанрів і підготуйте запит:

```
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;
using Microsoft.AspNetCore.Mvc;              // для BindProperty
using Microsoft.AspNetCore.Mvc.Rendering;   // для SelectList

namespace RazorPagesMovie.Pages.Catalog
{
    [AllowAnonymous]
    public class IndexModel : PageModel
    {
        private readonly RazorPagesMovieContext _context;
        public IndexModel(RazorPagesMovieContext context) => _context = context;

        public IList<Movie> Movies { get; private set; } = new List<Movie>();

        // Пошук за назвою (вже було)
        [BindProperty(SupportsGet = true)]
        public string? q { get; set; }

        // Нове: вибраний жанр
        [BindProperty(SupportsGet = true)]
        public string? genre { get; set; }

        // Нове: список жанрів для випадаючого списку
```

```
        public SelectList GenreOptions { get; private set; } = default!;

        public async Task OnGetAsync()
        {
            // 1) Підготувати список жанрів (без null/порожніх), унікальні,
відсортовані
            var genres = await _context.Movie.AsNoTracking()
                .Where(m => !string.IsNullOrEmpty(m.Genre))
                .Select(m => m.Genre!)
                .Distinct()
                .OrderBy(g => g)
                .ToListAsync();

            GenreOptions = new SelectList(genres);

            // 2) Базовий запит фільмів
            var query = _context.Movie.AsNoTracking().AsQueryable();

            if (!string.IsNullOrWhiteSpace(q))
            {
                // Пошук по назві (через LIKE для кращої сумісності)
                var pattern = $"%{q}%";
                query = query.Where(m => EF.Functions.Like(m.Title, pattern));
            }

            if (!string.IsNullOrWhiteSpace(genre))
            {
                // Точний збіг жанру
                query = query.Where(m => m.Genre == genre);
            }

            Movies = await query
                .OrderBy(m => m.Title)
                .ToListAsync();
        }
    }
}
```

Оновіть розмітку Pages/Catalog/Index.cshtml
Розширюємо форму пошуку полем «Жанр»:

```
@page
@model RazorPagesMovie.Pages.Catalog.IndexModel
@{
    ViewData["Title"] = "Каталог фільмів";
}

<h1 class="mb-3">@ViewData["Title"]</h1>

<form method="get" class="row g-2 mb-3 align-items-end">
    <div class="col-12 col-sm-6 col-md-5">
        <label for="q" class="form-label">Пошук за назвою</label>
        <input id="q" name="q" value="@Model.q" class="form-control"
placeholder="Напр., Matrix" />
```

```
    </div>

    <div class="col-12 col-sm-4 col-md-3">
        <label for="genre" class="form-label">Жанр</label>
        <select id="genre" asp-for="genre" asp-items="Model.GenreOptions" class="form-
select">
            <option value="">Усі жанри</option>
        </select>
    </div>

    <div class="col-12 col-sm-2 col-md-2">
        <button class="btn btn-primary w-100">Фільтрувати</button>
    </div>
    <div class="col-12 col-md-2">
        <a class="btn btn-outline-secondary w-100" asp-page="./Index">Скинути</a>
    </div>
</form>

@if (Model.Movies.Count == 0)
{
    <div class="alert alert-info">Нічого не знайдено.</div>
}
else
{
    <div class="text-muted small mb-2">
        Знайдено: <strong>@Model.Movies.Count</strong>
        @if (!string.IsNullOrWhiteSpace(Model.genre))
        {
            <span> • Жанр: <strong>@Model.genre</strong></span>
        }
        @if (!string.IsNullOrWhiteSpace(Model.q))
        {
            <span> • Пошук: <strong>@Model.q</strong></span>
        }
    </div>

    <div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-4 g-4">
        @foreach (var m in Model.Movies)
        {
            <div class="col">
                <div class="card h-100 shadow-sm border-0">
                    <img
                        src="~/images/placeholder-poster.svg"
                        class="card-img-top"
                        alt="Постер @m.Title"
                        style="object-fit:cover; aspect-ratio:2/3;" />
                    <div class="card-body d-flex flex-column">
                        <h5 class="card-title mb-1">@m.Title</h5>
                        <div class="text-muted small">@m.Genre</div>
                        <div class="mt-2 small">Дата релізу:
<strong>@m.ReleaseDate.ToString("yyyy-MM-dd")</strong></div>

                        @if (m.GetType().GetProperty("Price") != null)
                        {
```

```
                            <div class="mt-1 fw-semibold">Ціна: @((m as
dynamic).Price?.ToString("C"))</div>
                        }

                        <div class="mt-auto pt-2">
                            <a class="stretched-link" asp-page="./Details" asp-route-
id="@m.Id">Детальніше</a>
                        </div>
                    </div>
                </div>
            </div>
        }
    </div>
}
```

Пункт «Усі жанри» повертає порожнє значення genre, тому фільтр не застосовується.

Додамо фото (постер) до кожного фільму.

Найпростіший і надійний шлях: зберігаємо файли зображень у wwwroot/uploads/..., а в БД — лише відносний шлях (PosterPath). Завантаження/заміна — тільки через адмінські сторінки /Movies (які вже закриті роллю Admin). Публічні сторінки /Catalog показують реальний постер з fallback на плейсхолдер.

Підготовка (один раз)

- Переконайтеся, що Program.cs має app.UseStaticFiles();
- Створіть папку: wwwroot/uploads.

Оновлюємо модель Movie
Models/Movie.cs — додайте поле шляху до постера:

```
public string? PosterPath { get; set; }  // відносний шлях типу
"/uploads/movies/12/poster-abc123.webp"
```

Міграція (контекст RazorPagesMovieContext):

```
dotnet ef migrations add MoviePoster -c RazorPagesMovieContext -o Migrations
dotnet ef database update -c RazorPagesMovieContext
```

Admin: форма створення — поле «Постер»
Create.cshtml — додайте enctype і input

Pages/Movies/Create.cshtml

```
@page
@model RazorPagesMovie.Pages.Movies.CreateModel
@{
```

```
    ViewData["Title"] = "Create";
}
<h1>Create</h1>
//…
<form method="post" enctype="multipart/form-data">  <!-- ВАЖЛИВО -->
    <div class="row g-3">
        <!-- ваші поля Title/ReleaseDate/Genre/Price/Rating і т.д. -->

        <div class="col-12 col-md-6">
            <label class="form-label">Постер (jpg/png/webp, до 2 МБ)</label>
            <input asp-for="PosterFile" type="file" class="form-control"
accept=".jpg,.jpeg,.png,.webp" />
            <span class="text-danger" asp-validation-for="PosterFile"></span>
        </div>
    </div>

<div class="form-group">
    <input type="submit" value="Створити" class="btn btn-primary" />
</div>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Create.cshtml.cs — приймаємо файл і зберігаємо

Pages/Movies/Create.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using System.IO;

namespace RazorPagesMovie.Pages.Movies
{
    public class CreateModel : PageModel
    {
        private readonly RazorPagesMovie.Data.RazorPagesMovieContext _context;
        private readonly IWebHostEnvironment _env;

        public CreateModel(RazorPagesMovieContext context, IWebHostEnvironment env) //
<-- інжект env
        {
            _context = context;
            _env = env;
        }

        public IActionResult OnGet()
        {
```

```
            return Page();
        }

        [BindProperty]
        public Movie Movie { get; set; } = default!;

        [BindProperty]
        public IFormFile? PosterFile { get; set; }

        private static readonly string[] _permitted = { ".jpg", ".jpeg", ".png",
".webp" };
        private const long _maxSize = 2 * 1024 * 1024;

      //  public void OnGet() { }

        // To protect from overposting attacks, see https://aka.ms/RazorPagesCRUD
        public async Task<IActionResult> OnPostAsync()
        {
            if (!ModelState.IsValid)
            {
                return Page();
            }

            _context.Movie.Add(Movie);
            await _context.SaveChangesAsync();
            // Якщо додано файл — зберегти й оновити PosterPath
            if (PosterFile is not null && PosterFile.Length > 0)
            {
                try
                {
                    Movie.PosterPath = await SavePosterAsync(PosterFile, Movie.Id);
                    await _context.SaveChangesAsync();
                }
                catch (Exception ex)
                {
                    // за бажанням — видалити створений запис при фейлі завантаження
                    ModelState.AddModelError(string.Empty, $"Помилка завантаження
постера: {ex.Message}");
                    return Page();
                }
            }
            return RedirectToPage("./Index");
        }

        private async Task<string> SavePosterAsync(IFormFile file, int movieId)
        {
            var ext = Path.GetExtension(file.FileName).ToLowerInvariant();
            if (!_permitted.Contains(ext))
                throw new InvalidOperationException("Неприпустимий формат
зображення.");

            if (file.Length <= 0 || file.Length > _maxSize)
                throw new InvalidOperationException("Файл порожній або перевищує 2
МБ.");
```

```
            var destDir = Path.Combine(_env.WebRootPath, "uploads", "movies",
movieId.ToString());
            Directory.CreateDirectory(destDir);

            var fileName = $"poster-{Guid.NewGuid():N}{ext}";
            var filePath = Path.Combine(destDir, fileName);

            using (var stream = System.IO.File.Create(filePath))
                await file.CopyToAsync(stream);

            // веб-відносний шлях
            return $"/uploads/movies/{movieId}/{fileName}";
        }
    }
}
```

Admin: форма редагування — заміна постера (і видалення старого)
Edit.cshtml — додайте поле файла

Pages/Movies/Edit.cshtml

```
@page
@model RazorPagesMovie.Pages.Movies.EditModel

@{
    ViewData["Title"] = "Edit";
}

<h1>Edit</h1>

<h4>Movie</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form method="post" enctype="multipart/form-data">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <input type="hidden" asp-for="Movie.Id" />
            <div class="form-group">
                <label asp-for="Movie.Title" class="control-label"></label>
                <input asp-for="Movie.Title" class="form-control" />
                <span asp-validation-for="Movie.Title" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.ReleaseDate" class="control-label"></label>
                <input asp-for="Movie.ReleaseDate" class="form-control" />
                <span asp-validation-for="Movie.ReleaseDate" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Genre" class="control-label"></label>
                <input asp-for="Movie.Genre" class="form-control" />
                <span asp-validation-for="Movie.Genre" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Price" class="control-label"></label>
                <input asp-for="Movie.Price" class="form-control" />
                <span asp-validation-for="Movie.Price" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Rating" class="control-label"></label>
                <input asp-for="Movie.Rating" class="form-control" />
```

```
                <span asp-validation-for="Movie.Rating" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label class="form-label d-block">Поточний постер</label>
                <img        src="@(string.IsNullOrEmpty(Model.Movie.PosterPath)        ?
Url.Content("~/images/placeholder-poster.svg") : Url.Content(Model.Movie.PosterPath))"
                     class="img-fluid  rounded  border"  style="max-width:200px;  object-
fit:cover; aspect-ratio:2/3;" />
            </div>
            <div class="form-group">
                <label class="form-label">Змінити постер (jpg/png/webp, до 2 МБ)</label>
                <input     asp-for="PosterFile"     type="file"     class="form-control"
accept=".jpg,.jpeg,.png,.webp" />
                <span class="text-danger" asp-validation-for="PosterFile"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Зберегти" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>

<div>
    <a asp-page="./Index">Назад до списку</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Edit.cshtml.cs — логіка заміни

Pages/Movies/Edit.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using System.IO;

namespace RazorPagesMovie.Pages.Movies
{
    public class EditModel : PageModel
    {
        private readonly RazorPagesMovie.Data.RazorPagesMovieContext _context;
        private readonly IWebHostEnvironment _env;

        public EditModel(RazorPagesMovieContext context, IWebHostEnvironment env) // +
env
        {
            _context = context;
            _env = env;
        }
```

```
        [BindProperty]
        public Movie Movie { get; set; } = default!;

        [BindProperty]
        public IFormFile? PosterFile { get; set; }

        private static readonly string[] _permitted = { ".jpg", ".jpeg", ".png", ".webp" };
        private const long _maxSize = 2 * 1024 * 1024;

        public async Task<IActionResult> OnGetAsync(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var movie =  await _context.Movie.FirstOrDefaultAsync(m => m.Id == id);
            if (movie == null)
            {
                return NotFound();
            }
            Movie = movie;
            return Page();
        }

        // To protect from overposting attacks, enable the specific properties you want to bind to.
        // For more details, see https://aka.ms/RazorPagesCRUD.
        public async Task<IActionResult> OnPostAsync()
        {
            if (!ModelState.IsValid)
            {
                return Page();
            }

           // _context.Attach(Movie).State = EntityState.Modified;

            var dbMovie = await _context.Movie.FirstOrDefaultAsync(m => m.Id == Movie.Id);
            if (dbMovie == null) return NotFound();

            // Оновлюємо лише дозволені поля
            dbMovie.Title = Movie.Title;
            dbMovie.ReleaseDate = Movie.ReleaseDate;
            dbMovie.Genre = Movie.Genre;
            dbMovie.Price = Movie.Price;
            dbMovie.Rating = Movie.Rating;

            if (PosterFile is not null && PosterFile.Length > 0)
            {
                // завантажити новий
                var newPath = await SavePosterAsync(PosterFile, dbMovie.Id);

                // видалити старий (не обов'язково; але добре для чистоти)
                TryDeleteFile(dbMovie.PosterPath);

                dbMovie.PosterPath = newPath;
            }

            try
            {
                await _context.SaveChangesAsync();
            }
```

```
            catch (DbUpdateConcurrencyException)
            {
                if (!MovieExists(Movie.Id))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }

            return RedirectToPage("./Index");
        }

        private bool MovieExists(int id)
        {
            return _context.Movie.Any(e => e.Id == id);
        }

        private async Task<string> SavePosterAsync(IFormFile file, int movieId)
        {
            var ext = Path.GetExtension(file.FileName).ToLowerInvariant();
            if (!_permitted.Contains(ext))
                throw new InvalidOperationException("Неприпустимий формат зображення.");

            if (file.Length <= 0 || file.Length > _maxSize)
                throw new InvalidOperationException("Файл порожній або перевищує 2 МБ.");

            var   destDir   =   Path.Combine(_env.WebRootPath,   "uploads",   "movies",
movieId.ToString());
            Directory.CreateDirectory(destDir);

            var fileName = $"poster-{Guid.NewGuid():N}{ext}";
            var filePath = Path.Combine(destDir, fileName);

            using (var stream = System.IO.File.Create(filePath))
                await file.CopyToAsync(stream);

            return $"/uploads/movies/{movieId}/{fileName}";
        }

        private void TryDeleteFile(string? webPath)
        {
            if (string.IsNullOrWhiteSpace(webPath)) return;
            var           fullPath           =           Path.Combine(_env.WebRootPath,
webPath.TrimStart('/').Replace('/', Path.DirectorySeparatorChar));
            if (System.IO.File.Exists(fullPath))
            {
                try { System.IO.File.Delete(fullPath); } catch { /* ігноруємо */ }
            }
        }

    }
}
```

(Опційно) Видаляти постери при Delete

Pages/Movies/Delete.cshtml.cs — після Remove можете очистити папку:

```
// відразу після private readonly RazorPagesMovie.Data.RazorPagesMovieContext _context;
private readonly IWebHostEnvironment _env;
```

```
public DeleteModel(RazorPagesMovie.Data.RazorPagesMovieContext context,
IWebHostEnvironment env)
 {
     _context = context;
     _env = env ?? throw new ArgumentNullException(nameof(env));
 }

// додаємо в класі метод
private void TryDeletePosterFolder(int movieId)
{
    var dir = Path.Combine(_env.WebRootPath, "uploads", "movies", movieId.ToString());
    if (Directory.Exists(dir))
    {
        try { Directory.Delete(dir, recursive: true); } catch { }
    }
}

// в методі OnPostAsync після успішного SaveChanges:
TryDeletePosterFolder(Movie.Id);
```

Публічні сторінки: показуємо реальний постер

Catalog/Index.cshtml

Замість фіксованого SVG — умова з fallback:

```
<img
 src="@(string.IsNullOrEmpty(m.PosterPath) ? Url.Content("~/images/placeholder-poster.svg") :
Url.Content(m.PosterPath))"
 class="card-img-top"
 alt="Постер @m.Title"
 style="object-fit:cover; aspect-ratio:2/3;" />
```

Catalog/Details.cshtml

Те саме:

```
<img
 src="@(string.IsNullOrEmpty(Model.Movie?.PosterPath) ? Url.Content("~/images/placeholder-poster.svg") :
Url.Content(Model.Movie!.PosterPath))"
 alt="Постер @Model.Movie?.Title"
 class="img-fluid rounded shadow-sm"
 style="object-fit:cover; aspect-ratio:2/3;" />
```

(За бажанням — покажіть мініатюру в адмінському Movies/Index.)

Перевірка

1. Увійдіть як Admin → /Movies/Create → створіть фільм із постером.
2. Відкрийте /Catalog та /Catalog/Details/{id} — має показуватись саме ваш файл.
3. Через /Movies/Edit завантажте інший постер — перевірте заміну.
4. (Опційно) Видаліть фільм у /Movies/Delete — переконайтесь, що файли/папка зникли.

Рейтинги і коментарі

Зробимо так, щоб **кожен автентифікований користувач міг залишити ОДНУ оцінку (1–5) та коментар** до фільму (з можливістю оновити або видалити свій відгук), а на сторінці фільму показувались середня оцінка та список відгуків.

Models/Review.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace RazorPagesMovie.Models
{
    public class Review
    {
        public int Id { get; set; }

        [Required]
        public int MovieId { get; set; }

        // Identity користувача (з ApplicationDbContext)
        [Required, StringLength(450)]
        public string UserId { get; set; } = string.Empty;

        // Знімок відображуваного імені (щоб не тягнути UserManager для кожного)
        [StringLength(128)]
        public string? ReviewerName { get; set; }

        [Range(1, 5)]
        public int Rating { get; set; }

        [StringLength(2000)]
        public string? Comment { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
        public DateTime? UpdatedAt { get; set; }

        // Навігація
        public Movie? Movie { get; set; }
    }
}
```

У **Models/Movie.cs** додайте навігаційну колекцію:

public ICollection<Review> Reviews { get; set; } = new List<Review>();

Оновити DbContext

У **Data/RazorPagesMovieContext.cs**:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
```

```
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Data
{
    public class RazorPagesMovieContext : DbContext
    {
        public RazorPagesMovieContext (DbContextOptions<RazorPagesMovieContext>
options)
            : base(options)
        {
        }

        public DbSet<RazorPagesMovie.Models.Movie> Movie { get; set; } = default!;
        public DbSet<Review> Reviews { get; set; } = default!;   // ← ДОДАЛИ

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Один відгук на користувача на фільм
            modelBuilder.Entity<Review>()
                .HasIndex(r => new { r.MovieId, r.UserId })
                .IsUnique();

            modelBuilder.Entity<Review>()
                .HasOne(r => r.Movie)
                .WithMany(m => m.Reviews)
                .HasForeignKey(r => r.MovieId)
                .OnDelete(DeleteBehavior.Cascade);
        }
    }
}
```

Міграція БД (контекст RazorPagesMovieContext)

```
dotnet ef migrations add Reviews_Add -c RazorPagesMovieContext -o Migrations
dotnet ef database update -c RazorPagesMovieContext
```

Публічна сторінка деталей: показ і форма

Pages/Catalog/Details.cshtml.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Pages.Catalog
{
    [AllowAnonymous]
    public class DetailsModel : PageModel
    {
        private readonly RazorPagesMovieContext _context;
        private readonly UserManager<IdentityUser> _userManager;

        public DetailsModel(RazorPagesMovieContext context, UserManager<IdentityUser>
userManager)
        {
```

```
            _context = context;
            _userManager = userManager;
        }

        public Movie? Movie { get; private set; }
        public IList<Review> Reviews { get; private set; } = new List<Review>();
        public double AverageRating { get; private set; }
        public int ReviewsCount => Reviews.Count;

        // Для форми
        [BindProperty(SupportsGet = true)]
        public int id { get; set; }   // route id

        [BindProperty]
        public int? MyRating { get; set; }

        [BindProperty]
        public string? MyComment { get; set; }

        public async Task<IActionResult> OnGetAsync(int id)
        {
            this.id = id;
            Movie = await _context.Movie.AsNoTracking()
                .Include(m => m.Reviews)
                .FirstOrDefaultAsync(m => m.Id == id);
            if (Movie == null) return NotFound();

            Reviews = Movie.Reviews
                .OrderByDescending(r => r.UpdatedAt ?? r.CreatedAt)
                .ToList();

            AverageRating = Reviews.Count > 0 ? Reviews.Average(r => r.Rating) : 0;

            if (User.Identity?.IsAuthenticated == true)
            {
                var userId = _userManager.GetUserId(User);
                var mine = Reviews.FirstOrDefault(r => r.UserId == userId);
                if (mine != null)
                {
                    MyRating = mine.Rating;
                    MyComment = mine.Comment;
                }
            }

            return Page();
        }

        // Додати/оновити власний відгук
        [Authorize]
        public async Task<IActionResult> OnPostSaveReviewAsync()
        {
            if (id <= 0) return NotFound();
            if (MyRating is null || MyRating < 1 || MyRating > 5)
            {
                ModelState.AddModelError(nameof(MyRating), "Оцінка повинна бути від 1
до 5.");
                await OnGetAsync(id);
                return Page();
            }

            var movie = await _context.Movie.FirstOrDefaultAsync(m => m.Id == id);
            if (movie == null) return NotFound();

            var user = await _userManager.GetUserAsync(User);
```

```
            var userId = user!.Id;
            var displayName = user.Email ?? user.UserName ?? "user";

            var existing = await _context.Reviews.FirstOrDefaultAsync(r => r.MovieId ==
id && r.UserId == userId);
            if (existing == null)
            {
                _context.Reviews.Add(new Review
                {
                    MovieId = id,
                    UserId = userId,
                    ReviewerName = displayName,
                    Rating = MyRating.Value,
                    Comment = string.IsNullOrWhiteSpace(MyComment) ? null :
MyComment.Trim()
                });
            }
            else
            {
                existing.Rating = MyRating.Value;
                existing.Comment = string.IsNullOrWhiteSpace(MyComment) ? null :
MyComment.Trim();
                existing.ReviewerName = displayName;
                existing.UpdatedAt = DateTime.UtcNow;
            }

            await _context.SaveChangesAsync();
            return RedirectToPage("./Details", new { id });
        }

        // Видалити власний відгук (або адмін може видаляти будь-який)
        [Authorize]
        public async Task<IActionResult> OnPostDeleteReviewAsync(int reviewId)
        {
            var review = await _context.Reviews.FirstOrDefaultAsync(r => r.Id ==
reviewId);
            if (review == null) return NotFound();

            var isOwner = User.Identity?.IsAuthenticated == true &&
                          (await _userManager.GetUserIdAsync(await
_userManager.GetUserAsync(User))) == review.UserId;
            var isAdmin = User.IsInRole("Admin");

            if (!isOwner && !isAdmin) return Forbid();

            _context.Reviews.Remove(review);
            await _context.SaveChangesAsync();

            return RedirectToPage("./Details", new { id = review.MovieId });
        }
    }
}
```

Pages/Catalog/Details.cshtml

```
@page "{id:int}"
@model RazorPagesMovie.Pages.Catalog.DetailsModel
@{
    ViewData["Title"] = Model.Movie?.Title ?? "Фільм";
}

<a asp-page="./Index" class="btn btn-link ps-0">&larr; До каталогу</a>

<div class="row g-4 align-items-start">
```

```
    <div class="col-12 col-md-4 col-lg-3">
        <img src="@(string.IsNullOrEmpty(Model.Movie?.PosterPath) ?
Url.Content("~/images/placeholder-poster.svg") : Url.Content(Model.Movie!.PosterPath))"
             alt="Постер @Model.Movie?.Title"
             class="img-fluid rounded shadow-sm"
             style="object-fit:cover; aspect-ratio:2/3;" />
    </div>
    <div class="col">
        <h1 class="mb-2">@Model.Movie?.Title</h1>
        <div class="text-muted mb-3">@Model.Movie?.Genre</div>

        <dl class="row">
            <dt class="col-sm-3">Дата релізу</dt>
            <dd class="col-sm-9">@Model.Movie?.ReleaseDate.ToString("yyyy-MM-dd")</dd>

            @* Якщо є Price у моделі *@
            @if (Model.Movie?.GetType().GetProperty("Price") != null)
            {
                <dt class="col-sm-3">Ціна</dt>
                <dd class="col-sm-9">@((Model.Movie as
dynamic).Price?.ToString("C"))</dd>
            }

            @* Якщо є Rating у моделі *@
            @if (Model.Movie?.GetType().GetProperty("Rating") != null)
            {
                <dt class="col-sm-3">Рейтинг (текст)</dt>
                <dd class="col-sm-9">@((Model.Movie as dynamic).Rating)</dd>
            }
        </dl>

        <hr class="my-4" />

        <h3 class="mb-3">Відгуки</h3>

        @if (User.Identity?.IsAuthenticated == true)
        {
            <form method="post" asp-page-handler="SaveReview" class="mb-4">
                <input type="hidden" name="id" value="@Model.id" />
                <div class="mb-2">
                    <label class="form-label">Ваша оцінка</label>
                    <div class="d-flex gap-2">
                        @for (int i = 1; i <= 5; i++)
                        {
                            <div class="form-check form-check-inline">
                                <input class="form-check-input" type="radio" id="rate-
@i" name="MyRating" value="@i" @(Model.MyRating == i ? "checked" : "") />
                                <label class="form-check-label" for="rate-
@i">@i</label>
                            </div>
                        }
                    </div>
                    <span class="text-danger" asp-validation-for="MyRating"></span>
                </div>

                <div class="mb-2">
                    <label class="form-label" for="MyComment">Коментар
(необов’язково)</label>
                    <textarea id="MyComment" name="MyComment" class="form-control"
rows="3">@Model.MyComment</textarea>
                </div>

                <button class="btn btn-primary">Зберегти відгук</button>
            </form>
```

```
        }
        else
        {
            <div class="alert alert-info">
                <a asp-area="Identity" asp-page="/Account/Login" asp-route-
ReturnUrl="@Url.Page("./Details", new { id = Model.id })">Увійдіть</a>, щоб залишити
відгук.
            </div>
        }

        @if (Model.ReviewsCount == 0)
        {
            <div class="text-muted">Ще немає відгуків.</div>
        }
        else
        {
            <div class="list-group">
                @foreach (var r in Model.Reviews)
                {
                    <div class="list-group-item">
                        <div class="d-flex justify-content-between">
                            <div>
                                <strong>@(string.IsNullOrWhiteSpace(r.ReviewerName) ?
"Користувач" : r.ReviewerName)</strong>
                                <span class="ms-2">
                                    @for (int i = 1; i <= 5; i++)
                                    {
                                        @:(@(i <= r.Rating ? "★" : "☆"))
                                    }
                                </span>
                            </div>
                            <small class="text-muted">
                                @((r.UpdatedAt ??
r.CreatedAt).ToLocalTime().ToString("yyyy-MM-dd HH:mm"))
                            </small>
                        </div>
                        @if (!string.IsNullOrWhiteSpace(r.Comment))
                        {
                            <div class="mt-2">@r.Comment</div>
                        }

                        @* Кнопка видалити — власник або адмін *@
                        @if ((User.Identity?.IsAuthenticated ?? false) &&
                                        (User.IsInRole("Admin") ||
User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value == r.UserId))
                        {
                            <form method="post" asp-page-handler="DeleteReview"
class="mt-2">
                                <input type="hidden" name="reviewId" value="@r.Id" />
                                <button class="btn btn-sm btn-outline-danger"
type="submit">Видалити</button>
                            </form>
                        }
                    </div>
                }
            </div>
        }
        @* Тут пізніше додамо: фото з БД/файлів, оцінювання, коментарі *@
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Контрольні питання

1. Що таке Razor Pages?
2. Де зберігаються сторінки Razor?
3. За що відповідає файл _Layout.cshtml?
4. Для чого папка wwwroot/?
5. Як додати таблицю/колонку в БД через EF Core?
6. Що таке міграція?
7. Як обмежити доступ до папки сторінок лише для ролі Admin?
8. Де зберігаються завантажені зображення?